

Projet EFREI - Module TI616



Valentin GONCALVES, Batur HAMZAOGULLARI, Emma DUVERNET, Ivane DJOTEBONG
TIDONG, Roline IMELE TIODA

Repo github:

[valentino3112/recette-et-avis: Plateforme communautaire de recettes et d'avis — EFREI TI616](https://github.com/valentino3112/recette-et-avis)

SOMMAIRE

SOMMAIRE.....	2
Présentation du projet.....	3
Architecture et conception.....	3
Choix technologiques justifiés.....	8
Implementation.....	9
Analyse de l’empreinte carbone.....	12
Tests et validations.....	18
Organisation de l’équipe et collaboration.....	20
Discussion et conclusion.....	21
Annexes.....	22

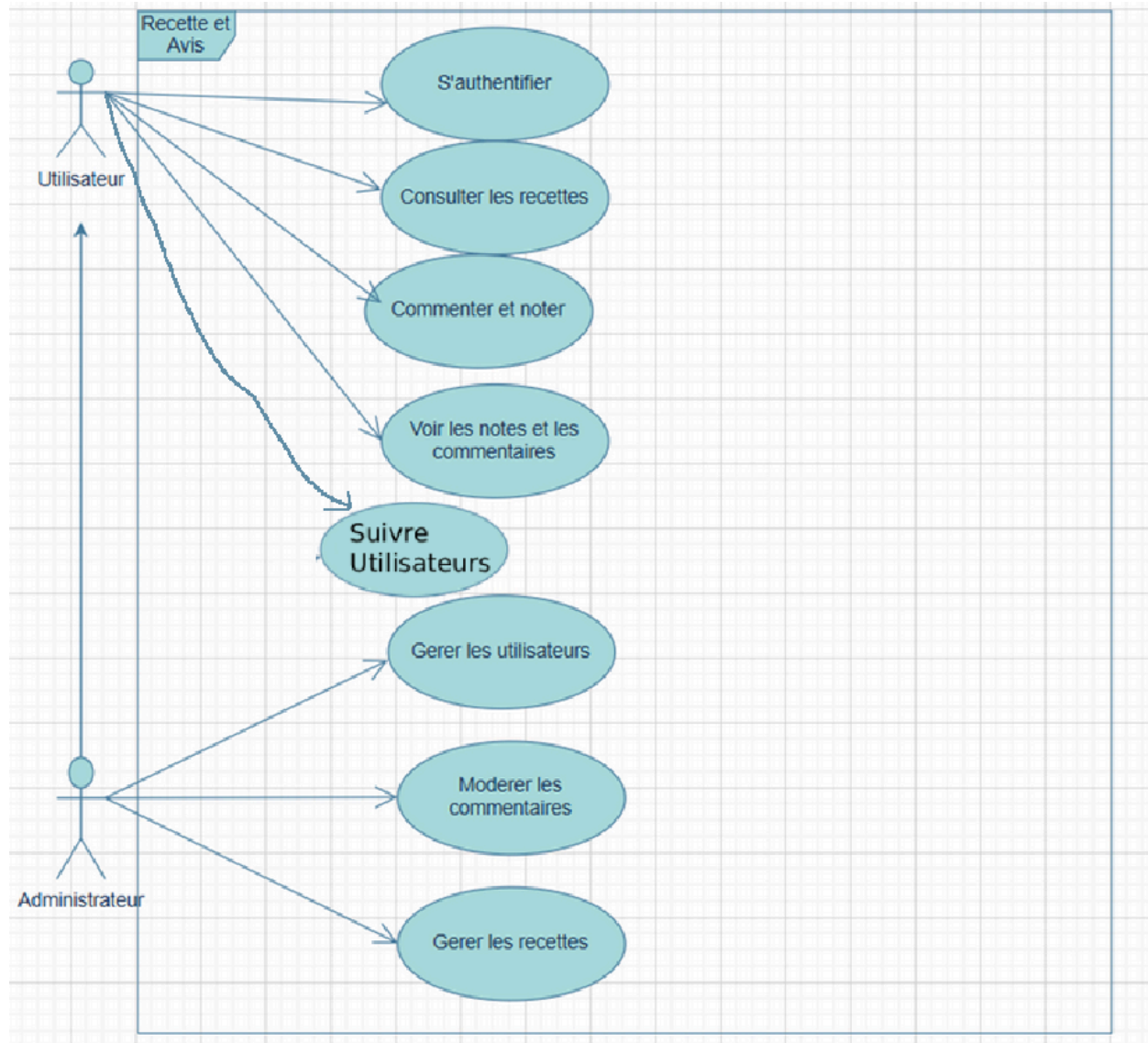
Présentation du projet

Recette & Avis est un site web qui permet de consulter des recettes de cuisine et de lire les avis d'autres utilisateurs. Sur internet il existe beaucoup de recettes mais il est souvent difficile de savoir si elles sont vraiment bonnes. Notre site répond à ce besoin en permettant aux membres de noter et commenter chaque recette, pour aider les autres à faire le bon choix. Cependant l'objectif principal du projet va au-delà du service rendu : il s'agit de concevoir un site web fonctionnel tout en ayant le plus faible impact possible sur l'environnement, en appliquant les principes de l'éco-conception numérique. Les fonctionnalités principales sont la création de compte, la consultation des recettes, l'ajout de notes et de commentaires, la gestion de son profil et un espace administrateur pour gérer le contenu. Le site s'adresse principalement aux étudiants, aux jeunes adultes et aux personnes qui cuisinent de temps en temps. Il existe trois types d'utilisateurs : le visiteur qui peut consulter les recettes sans créer de compte, l'utilisateur connecté qui peut noter et commenter, et l'administrateur qui gère le contenu du site. Chaque décision technique a été pensée dans une logique écologique : pas d'images inutiles, pas de bibliothèques lourdes, des polices déjà présentes sur l'ordinateur de l'utilisateur, une base de données légère sans serveur dédié et des listes affichées par petits groupes plutôt que tout d'un coup.

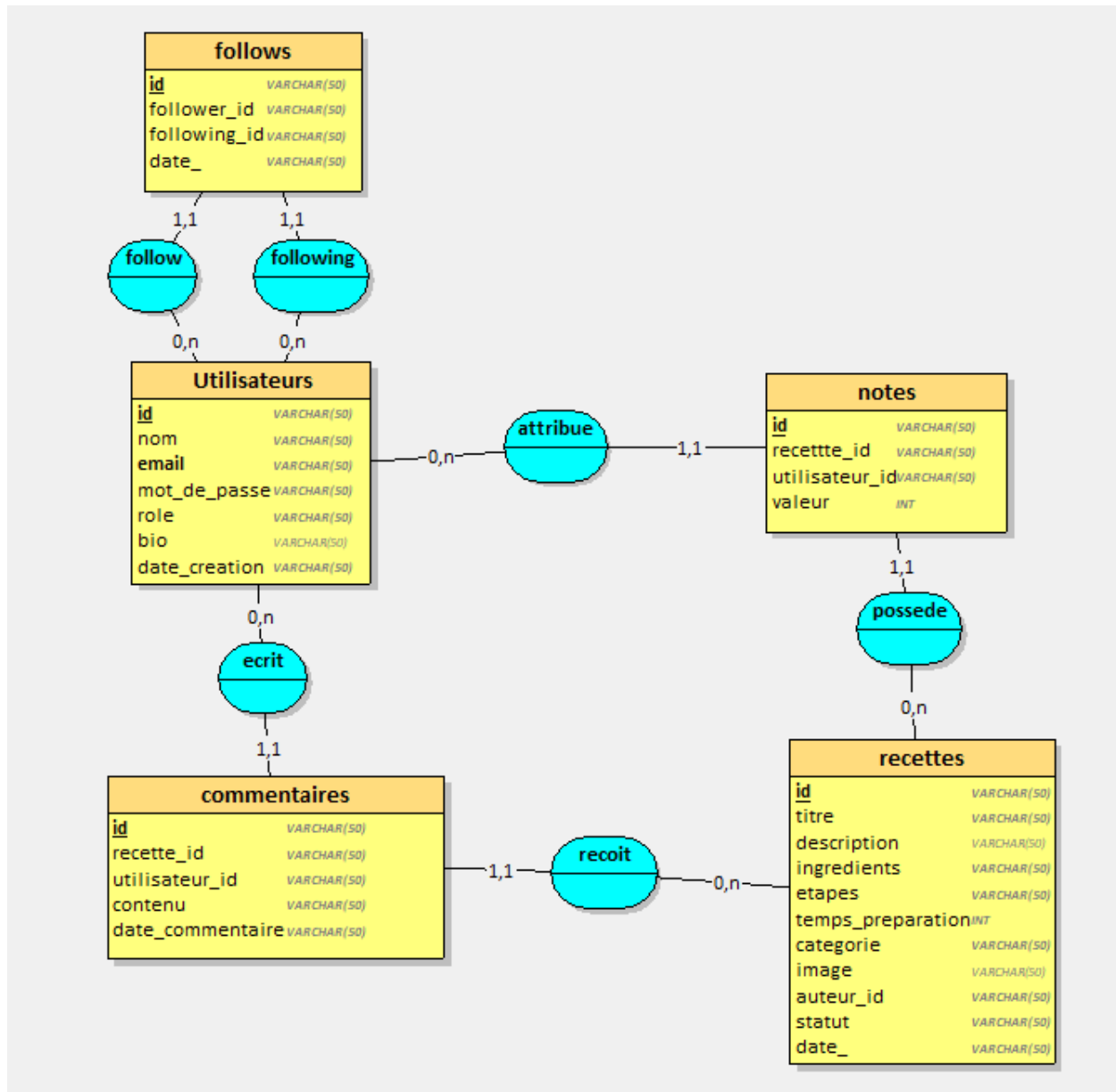
Architecture et conception

Le projet repose sur une architecture de trois niveaux. Le front-end est développé en HTML, CSS et JavaScript . Le back-end est développé en Node.js avec Express, qui gère toutes les routes et la logique du serveur et puis la base de donnée avec SQLite. Trois diagrammes UML ont été réalisés lors de la phase de conception afin de modéliser le fonctionnement du système, les entités de la base de données et les interactions entre les différents acteurs.

Tout d'abord le diagramme de cas d'utilisation qui identifie les deux acteurs du système : l'utilisateur connecté et l'administrateur, ainsi que les actions que chacun peut effectuer sur la plateforme.



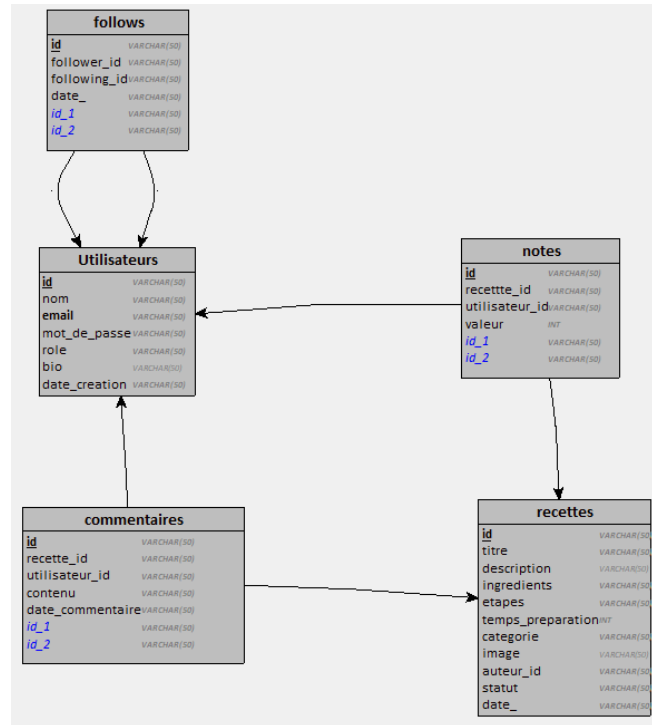
Ensuite le diagramme de classes qui représente les quatre entités de la base de données, à savoir l'utilisateur, la recette, le commentaire et la note, avec leurs attributs et leurs relations.



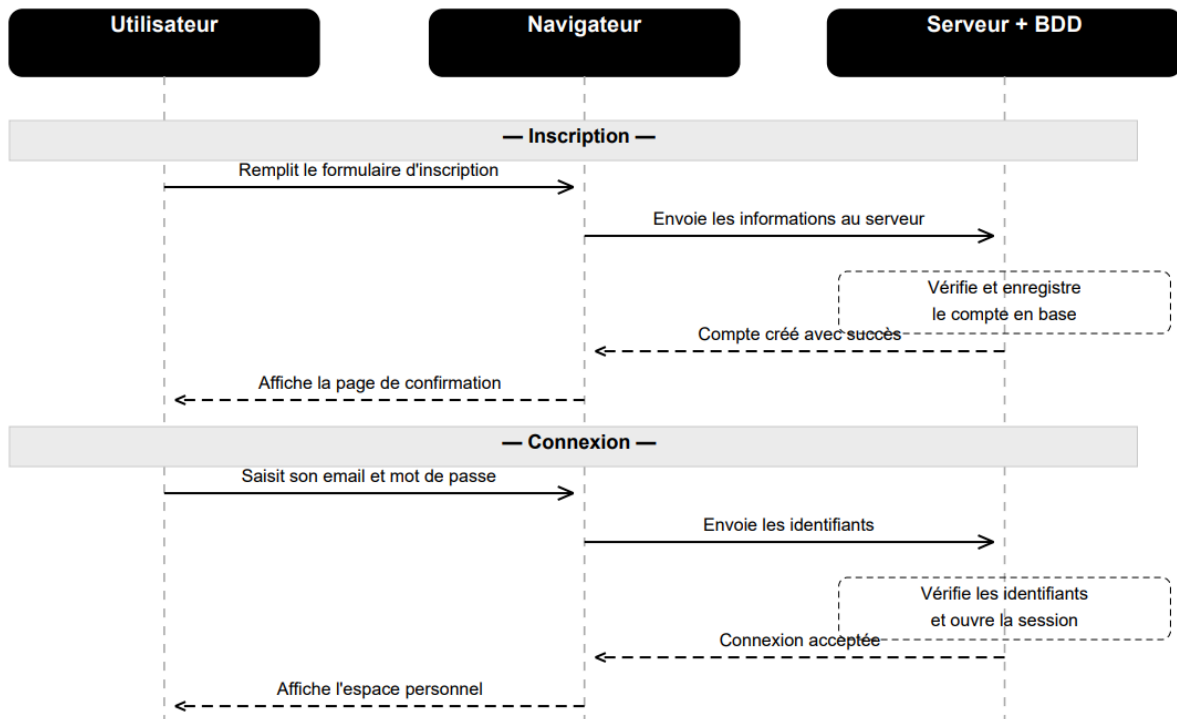
MLD

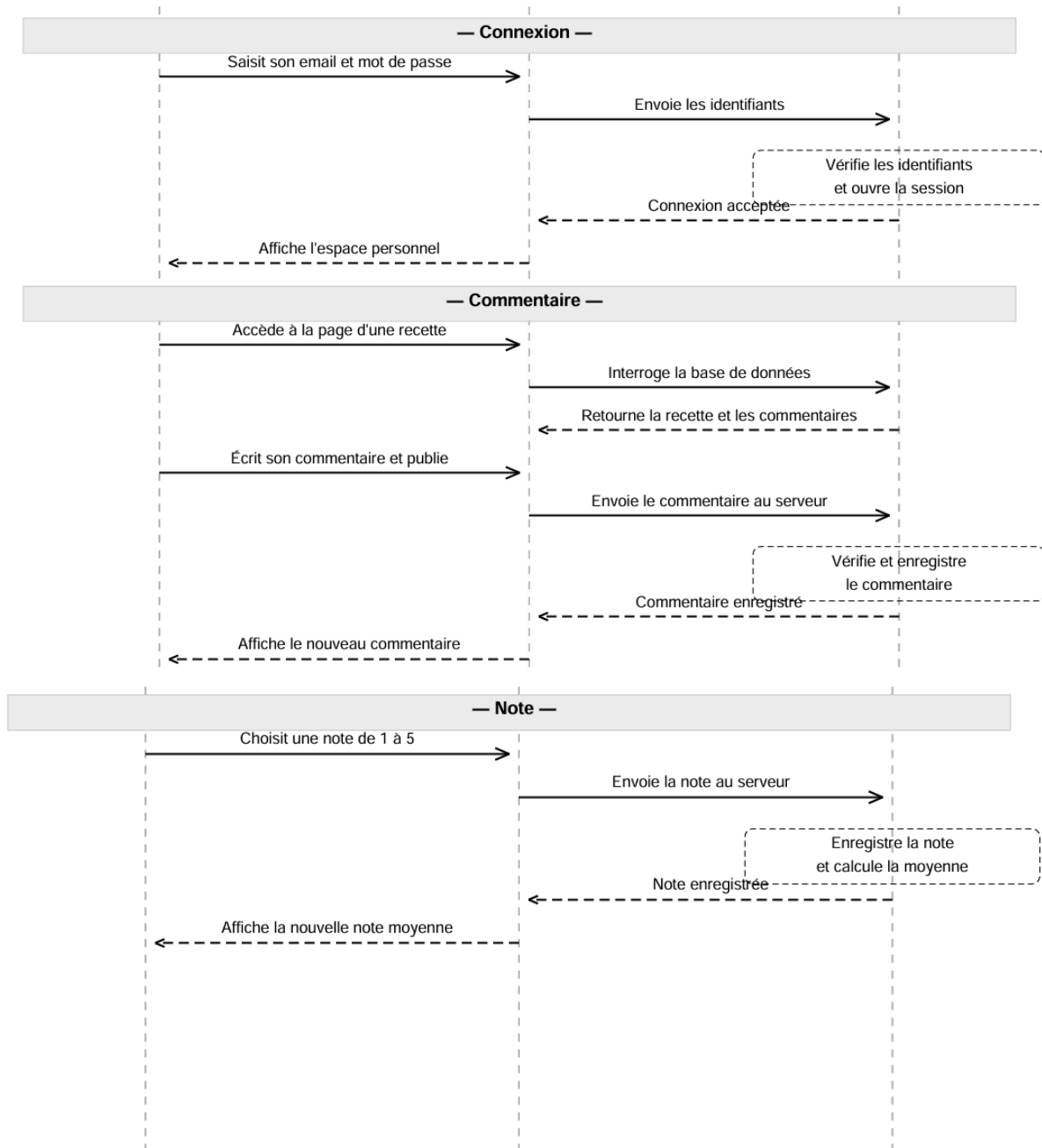
```
Utilisateurs = (id VARCHAR(50), nom VARCHAR(50), email VARCHAR(50), mot_de_passe VARCHAR(50), role VARCHAR(50), bio VARCHAR(50), date_creation VARCHAR(50));
recettes = (id VARCHAR(50), titre VARCHAR(50), description VARCHAR(50), ingredients VARCHAR(50), etapes VARCHAR(50), temps_preparation INT, categorie VARCHAR(50), image VARCHAR(50), auteur_id VARCHAR(50), statut VARCHAR(50), date_ VARCHAR(50));
commentaires = (id VARCHAR(50), recette_id VARCHAR(50), utilisateur_id VARCHAR(50), contenu VARCHAR(50), date_commentaire VARCHAR(50), #id_1, #id_2);
notes = (id VARCHAR(50), recette_id VARCHAR(50), utilisateur_id VARCHAR(50), valeur INT, #utilisateur_id, #recette_id);
follows = (id VARCHAR(50), follower_id VARCHAR(50), following_id VARCHAR(50), date_ VARCHAR(50), #follower_id, #following_id);
```

Les clés étrangères ne sont pas bien modélisées sur le schéma MLD du looping donc nous vous fournissons aussi le code MLD modifié manuellement ci-dessus



Enfin le diagramme de séquence qui décrit étape par étape le parcours complet de l'utilisateur, depuis l'inscription et la connexion jusqu'à l'ajout d'un commentaire et d'une note sur une recette.





La base de données est composée de 5 tables:

Utilisateurs = (id VARCHAR(50), nom VARCHAR(50), email VARCHAR(50), mot_de_passe VARCHAR(50), role VARCHAR(50), bio VARCHAR(50), date_creation VARCHAR(50));

recettes = (id VARCHAR(50), titre VARCHAR(50), description VARCHAR(50), ingredients VARCHAR(50), etapes VARCHAR(50), temps_preparation INT, categorie VARCHAR(50), image VARCHAR(50), auteur_id VARCHAR(50), statut VARCHAR(50), date_ VARCHAR(50));

commentaires = (id VARCHAR(50), recette_id VARCHAR(50), utilisateur_id VARCHAR(50), contenu VARCHAR(50), date_commentaire VARCHAR(50), #id_1, #id_2);

notes = (id VARCHAR(50), recette_id VARCHAR(50), utilisateur_id VARCHAR(50), valeur INT, #utilisateur_id, #recette_id);

follows = (id VARCHAR(50), follower_id VARCHAR(50), following_id VARCHAR(50), date_ VARCHAR(50), #follower_id, #following_id);

Choix technologiques justifiés

Technologie	Role	Justification Green IT
Frontend React/Vite	Front-end	Développement par composants, build rapide, bundle minifié. Développement par composants, build rapide, bundle minifié.
Node.js + Express	Back-end	API REST simple, écosystème connu, middleware faciles à expliquer. Serveur léger pour un projet académique ; démarrage simple sur Railway.
SQLite	Base de données	Base fichier, schema relationnel simple, pas de serveur SQL externe.

		Moins d'infrastructure à maintenir ; appropriée pour un faible volume de données.
Git + GitHub	Collaboration	Il permet toute l'équipe de travailler sur le même code sans avoir besoin d'un serveur privé ou d'un outil payant. Tout est géré en ligne, gratuitement, sans consommer de ressources supplémentaires.
Format WebP	Images	Les images en WebP sont jusqu'à 30% plus légères qu'en JPEG ou PNG, donc moins de données à télécharger
Railway / Nixpacks	Déploiement	Configuration directe via railway.json et .nixpacks.toml. Ressources adaptées au besoin du service.

Implementation

L'implémentation de Recette & Avis repose sur une architecture simple en trois parties : un frontend en React/Vite, un backend en Node.js/Express et une base de données SQLite. Cette organisation permet de séparer l'interface utilisateur, la logique serveur et la persistance des données, tout en gardant une solution légère et facile à maintenir.

Le frontend permet aux visiteurs de consulter les recettes, de les filtrer, de voir le détail d'une recette et de lire les avis. Les utilisateurs connectés peuvent créer un compte, se connecter, proposer une recette, commenter et noter. Un espace administrateur permet de gérer les recettes, les commentaires et les utilisateurs.

Le backend expose une API REST organisée par domaine fonctionnel : authentification, utilisateurs, recettes, commentaires, notes et administration. Les données sont stockées dans SQLite, ce qui évite d'utiliser un serveur de base de données séparé et reste cohérent avec l'objectif d'éco-conception du projet.

Les appels au backend sont centralisés dans `frontend/js/api.js`. Cela évite de répéter du code fetch dans chaque composant React et facilite la maintenance.

```
const API_BASE = "/api";
async function request(path, options = {}) {
  const response = await fetch(`${API_BASE}${path}`, {
    credentials: "include",
    headers: { "Content-Type": "application/json" },
    ...options
  });
  const data = await response.json().catch(() => null);
  if (!response.ok) {
    throw new Error(data?.message || "Erreur lors de la requête");
  }
  return data;
}
```

L'option `credentials: "include"` permet d'envoyer les cookies de session au serveur. Ainsi, le backend peut identifier l'utilisateur connecté sans stocker de token dans le navigateur.

CRUD des recettes

La fonctionnalité principale du projet est la gestion des recettes. Le backend permet de récupérer, créer, modifier et supprimer des recettes grâce à des routes REST. Exemple simplifié de récupération des recettes :

```

router.get("/", (req, res) => {
  const recettes = db.prepare(`
    SELECT r.id, r.titre, r.description, r.categorie, r.image,
           u.nom AS auteur,
           ROUND(AVG(n.valeur), 1) AS note_moyenne,
           COUNT(DISTINCT c.id) AS total_commentaires
    FROM recettes r
    LEFT JOIN utilisateurs u ON r.utilisateur_id = u.id
    LEFT JOIN notes n ON n.recette_id = r.id
    LEFT JOIN commentaires c ON c.recette_id = r.id
    GROUP BY r.id
    ORDER BY r.date_creation DESC
    LIMIT ?
  `).all(50);

  res.json({ recettes });
});

```

Authentication

L'authentification repose sur des sessions serveur avec express-session. Lorsqu'un utilisateur se connecte, le backend vérifie son email et compare le mot de passe avec la version hashée stockée en base.

```

const user = db.prepare(`
  SELECT id, nom, email, mot_de_passe, role
  FROM utilisateurs
  WHERE email = ?
`).get(email);

const passwordOk = await bcrypt.compare(password, user.mot_de_passe);

if (passwordOk) {
  req.session.user = {
    id: user.id,
    nom: user.nom,
    email: user.email,
    role: user.role
  };
}

```

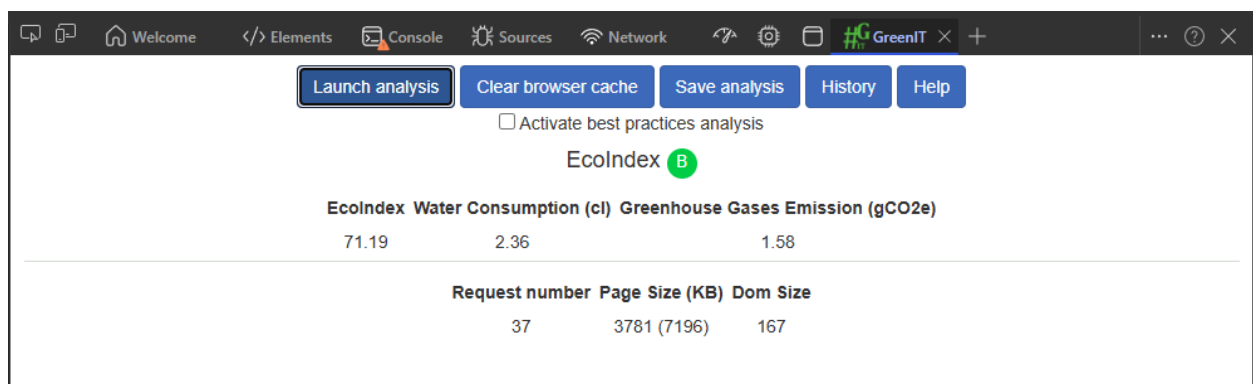
Analyse de l'empreinte carbone

Test avant optimisation

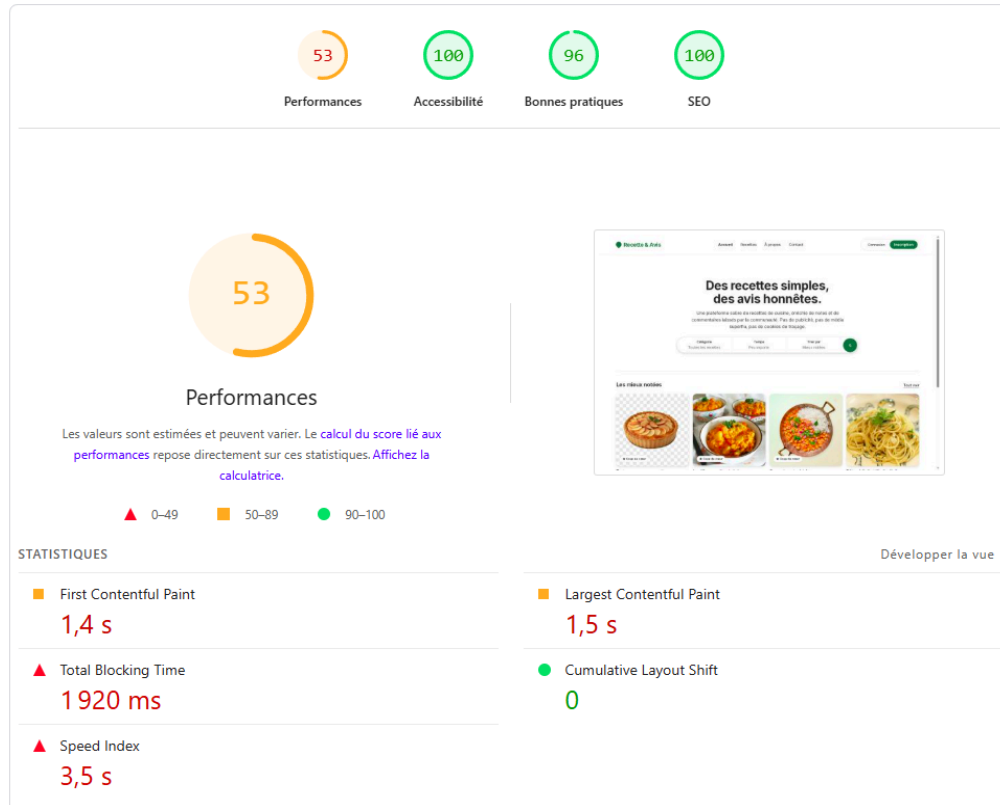
EcoIndex:



GreenITAnalysis :



Lighthouse:



Sources de pollution numérique identifiées

- Polices externes: chargement de Google Fonts (requête externe supplémentaire, payload).
- Chargement et transpilation côté client: usage de React/ReactDOM en build de développement + Babel standalone (transpilation JSX dans le navigateur) — très coûteux en taille et CPU, augmente le poids et le temps CPU côté client.
- Plusieurs scripts non différés: de nombreux scripts (y compris Babel) bloquaient le rendu.
- Format d'images non optimisé: images PNG (potentiellement grandes) — même si loading="lazy" est présent, les PNG sont moins efficaces que WebP/AVIF.

Optimisations appliquées

Polices systèmes:

- Quoi: suppression des liens vers Google Fonts dans index.html et mise à jour de la variable --sans dans styles.css pour n'utiliser que des polices système.
- Pourquoi: évite une requête réseau externe et le téléchargement de fichiers de polices, réduit le poids initial et l'empreinte CO2 par visite.
- Impact attendu: diminution des requêtes externes et du poids total par page.

React en version production + scripts différés:

- Quoi: remplacement des builds dev de React/ReactDOM par react.production.min.js et react-dom.production.min.js dans index.html ; ajout de l'attribut defer à tous les scripts (y compris Babel pour garder la compatibilité prototype).
- Pourquoi: les builds production sont minifiés et beaucoup plus légers ; defer évite le blocage du parsing HTML par les scripts, réduisant le temps jusqu'au rendu.
- Limite: tant que l'application est fournie en JSX et transpile dans le navigateur via Babel, il reste un coût important. Solution recommandée suivante: précompiler/bundler (voir recommandations).

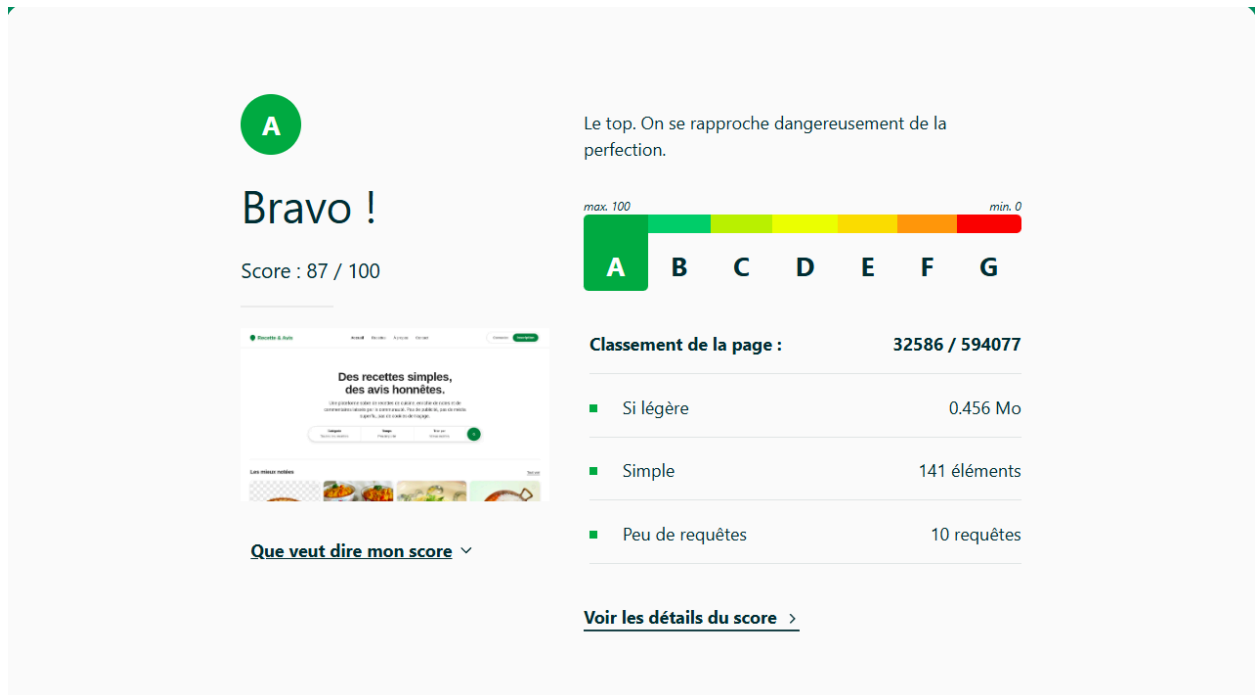
Décodage asynchrone des images:

- Quoi: ajout de decoding="async" sur les balises principales dans pages-public.jsx.
- Pourquoi: réduit le travail du thread principal lors du décodage d'images, améliore la réactivité et peut aider le LCP dans certains cas.
- Remarque: loading="lazy" était déjà présent, c'est bon.

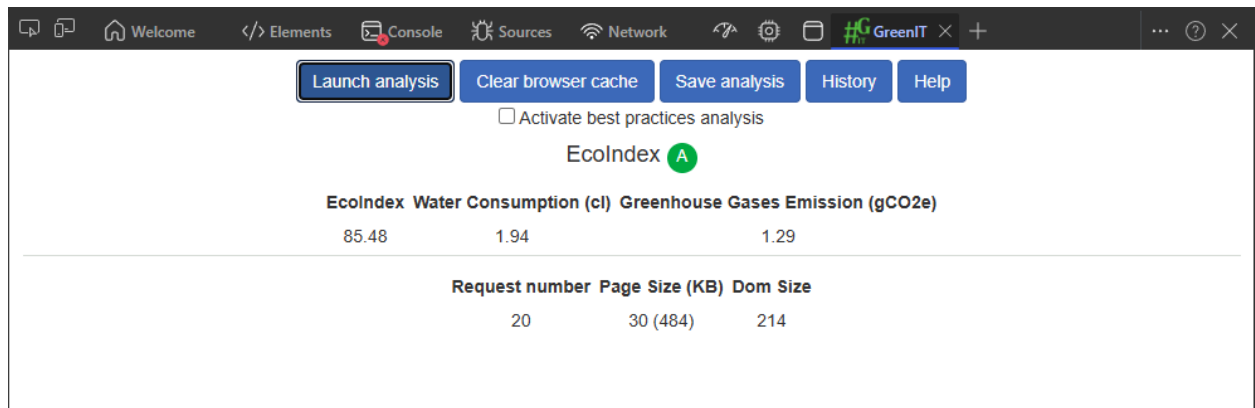
Conversion des PNG en WebP et servir via <picture> ou srcset. Idéal: générer images à plusieurs résolutions (responsive) pour réduire le poids moyen par visite.

Test après optimisation

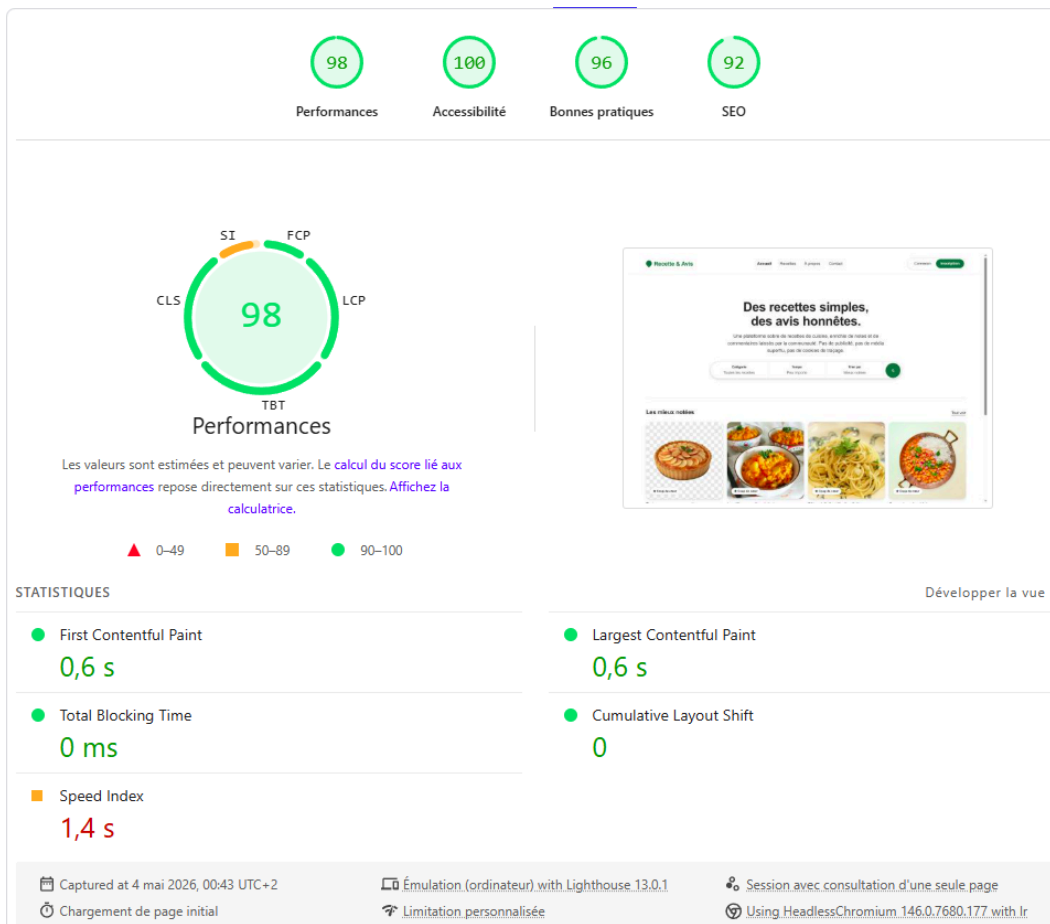
EcoIndex:



GreenITAnalysis :



Lighthouse:



Conclusion

Indicateur	Avant	Après	Gain
Poids de la page d'accueil	3781 Ko	456 Ko	88%
Nombre de requêtes HTTP	37	10	27
Score EcoIndex	74/100	87/100	13 pts
Note EcoIndex	B (A–G)	A (A–G)	
CO ₂ / visite (EcoIndex)	1.52 g	1.38 g	9%
Score Lighthouse Perf.	53/100	98/100	45 pts
FCP	1,4 s	0,6 s	57%
LCP	1,5 s	0,6 s	60%

Tests et validations

Scénario	Résultat attendu	Status
Créer un utilisateur valide	Utilisateur enregistré en BDD, redirection	ok
Créer un utilisateur (email vide	Message d'erreur côté serveur	ok
Modifier un utilisateur existant	Données mises à jour en BDD	ok
Supprimer un utilisateur	Utilisateur supprimé après confirmation	ok
Lister les utilisateurs	Affichage paginé correct	ok
Créer une entité métier	Entité enregistrée, liste mise à jour	
Connexion avec identifiants valides	Redirection vers espace personnel	ok
Connexion avec mauvais MDP	Message d'erreur, pas de session créée	ok
Accès page protégée sans connexion	Redirection vers la page de connexion	ok

Exemple de scénario:



[Accueil](#) [Recettes](#) [Proposer](#) [À propos](#) [Contact](#) [Profil](#)

Camille D.

Déconnexion

Mon profil

Nom

Camille D.

Email

camille@example.com

Rôle

Utilisateur

Inscrit le

01 mars 2026

Enregistrer

Mon activité

0

 recette

1

 abonné

2

 abonnements

Voir mon profil public →

Recette & Avis

[Accueil](#)[Recettes](#)[Proposer](#)[À propos](#)[Contact](#)[Profil](#)[Admin](#)

Admin EFREIADMINDéconnexion

Gestion des utilisateurs

[Tableau de bord](#)[Recettes](#)[Utilisateurs](#)[Commentaires](#)

2 comptes - liste paginée (LIMIT/OFFSET côté serveur)

NOM	EMAIL	RÔLE	INSCRIT LE	ACTIVITÉ	
Admin EFREI (vous)	admin@recetteavis.fr	ADMIN	01 mars 2026	0 cmt - 0 notes	→ user Suppr.
Camille D.	camille@example.com	USER	01 mars 2026	0 cmt - 0 notes	→ admin Suppr.

Recette & Avis

[Accueil](#)[Recettes](#)[À propos](#)[Contact](#)

ConnexionInscription

Accès restreint — connexion requise.

Se connecter

Organisation de l'équipe et collaboration

Valentin GONÇALVES

Frontend · architecture React ·
GitHub

Responsable du planning, des
wireframes et de l'intégration
HTML/CSS.

Batur HAMZAOGULLARI

Back-end · API Express

Implémentation des routes
Express, du hash bcrypt et des
sessions.

Emma DUVERNET

Base de données · migrations SQLite

Modélisation SQLite, scripts de
migration, requêtes paramétrées.

Ivane DJOTEBONG TIDONG

Tests · qualité & sécurité

Tests Jest/Supertest, audit
accessibilité, contrastes et
hiérarchie.

Roline IMELE TIODA

Green IT · déploiement

Mesures EcoIndex / Lighthouse,
optimisations Vite, déploiement
Railway.

Discussion et conclusion

Le projet Recette & Avis présente une base solide pour le projet : la finalité est claire, l'architecture est lisible, la stack est adaptée au périmètre et la démarche d'éco-conception est déjà présente. Le passage à Réact/Vite constitue une amélioration importante par rapport à une approche avec React/Babel charges directement dans le navigateur.

Les résultats Ecograder fournis indiquent un poids de page raisonnable, autour de 449,25 KB, avec une bonne minification et une compression active. Le principal axe technique d'optimisation reste le JavaScript inutilisé au chargement initial, ce qui peut être corrigé par du code splitting et du lazy loading.

Annexes

Démarrage local (mode développement)

```
# Installe les dépendances  
npm ci
```

```
# Lance le backend (port 3000) — initialise la BDD si vide  
npm run dev
```

```
# Dans un second terminal : lance le frontend Vite (port 5173)  
npm run dev:frontend
```

Build de production (vérification locale)

```
npm run build      # compile le frontend → dist/  
npm run start      # sert la version de production sur :3000
```

Base de données SQLite

```
# Réinitialiser complètement (supprime et recrée toutes les données fictives)  
npm run db:init
```

```
# Insérer les données seulement si la base est vide (utilisé au démarrage)  
npm run db:init-if-empty
```

```
# Supprimer la base pour repartir de zéro  
rm database/recettes.db
```